

REDROB HACKATHON

API Mode Implementation Guide

Upgrading from Local Models to API-Powered Intelligence

OpenAI · Google Gemini · Anthropic Claude · Cohere — Backend logic · New features · UI changes · Cost

0. Overview — What Changes When You Add an API Key

The system currently runs in two modes. This document explains how to build the full API mode alongside the existing local mode — not replacing it.

Aspect	Local Mode (current)	API Mode (this document)
Embedding	sentence-transformers all-MiniLM-L6-v2 (local 80MB model)	OpenAI text-embedding-3-small or Google text-embedding-004 (API call)
Re-ranking	cross-encoder/ms-marco-MiniLM-L-6-v2 (local 117MB model)	GPT-4o-mini / Gemini 1.5 Flash / Claude Haiku — reads candidate + JD and scores
Reasoning	Template-based — fills in title, YoE, company, skills from profile	LLM-generated — reads full career history and writes unique, nuanced 2-3 sentence reasoning
JD Parsing	Rule-based extraction from static job_description.docx	LLM reads any pasted JD and extracts requirements dynamically — works for any role
Conversational Features	Not available	New: ask questions about candidates, explain ranking decisions, compare candidates in natural language
Quality	Good — industry-standard IR pipeline	Significantly better — LLM understands context, nuance, and plain-language descriptions
Cost	Free (local compute only)	~\$0.05–\$1.05 per 100K run depending on provider
Competition compliance	Fully compliant (no network)	Sandbox/demo only — competition rank.py still uses local mode

KEY DECISION: Local mode and API mode coexist. The competition rank.py always uses local mode (no network allowed). The sandbox demo can use API mode for dramatically better quality. Users toggle between modes in the S0 API Settings screen or S5 Configuration.

1. API Configuration — What to Build

1.1 Supported Providers

Provider	Embedding Model	Re-ranking / LLM	Reasoning Model	Strengths
OpenAI	text-embedding-3-small (1536 dims, \$0.02/1M tokens)	GPT-4o-mini	GPT-4o-mini	Best JSON reliability, most consistent structured output, widely known
Google Gemini	text-embedding-004 (768 dims, free generous tier)	Gemini 1.5 Flash	Gemini 1.5 Flash	Fastest API, cheapest, excellent multilingual, 1M context window
Anthropic Claude	No embedding model available	Claude 3 Haiku	Claude 3 Haiku	Best at nuanced reasoning, excellent for grounded candidate explanations
Cohere	embed-english-v3.0 (1024 dims)	Command-R	Command-R	Purpose-built for retrieval, strong on technical content

RECOMMENDATION: Use Google Gemini as primary — text-embedding-004 is free for reasonable usage and Gemini 1.5 Flash is the cheapest re-ranking model. Use OpenAI as fallback — GPT-4o-mini has the most reliable JSON output for structured ranking responses.

1.2 Per-Task Model Assignment

Different tasks need different models. Assign independently:

Task	Best API Choice	Why	Fallback (local)
Embedding (100K candidates)	Google text-embedding-004	Free, fast, 768 dims — enough for this task. Saves cost vs OpenAI.	sentence-transformers all-MiniLM-L6-v2
Re-ranking top 300	Gemini 1.5 Flash	Fastest, cheapest, good at structured scoring with JSON output.	cross-encoder/ms-marco-MiniLM-L-6-v2

Task	Best API Choice	Why	Fallback (local)
Reasoning generation (top 100)	GPT-4o-mini or Claude Haiku	Best quality for nuanced, grounded 2-3 sentence professional reasoning.	Template-based builder
JD Parsing (dynamic)	GPT-4o or Gemini 1.5 Pro	JD parsing is one-time — worth using a stronger model for accuracy.	Rule-based extraction from hardcoded JD
Conversational Q&A	GPT-4o-mini or Gemini 1.5 Flash	Interactive chat — speed matters more than maximum quality.	Not available in local mode

1.3 API Key Storage Logic

- Store API keys in a .env file at the project root — never in source code
- Load with python-dotenv: `from dotenv import load_dotenv, os.getenv('OPENAI_API_KEY')`
- Support multiple providers simultaneously — user picks which to use per task in S0 settings
- In the sandbox demo frontend, accept API keys in the S0 settings screen and pass them in the Authorization header to your FastAPI backend
- Never log API keys. Never expose them in API responses. Never commit to GitHub.

2. Backend Module Changes — What Gets Upgraded

2.1 Module 4: Embedding Engine — API Version

Replace local sentence-transformers embedding with API embedding. The key difference: API embeddings are called once per candidate text, not pre-computed. This means you either pre-compute offline using the API (allowed) or embed on-demand (expensive for 100K).

Strategy: Hybrid Embedding

Scenario	Embedding Approach
Full 100K competition run	Pre-compute using API offline (one-time). Cache to .npy file. Same as local approach — just the model is API-based. Cost: ~\$2 for Google, ~\$10 for OpenAI for all 100K.
Demo run (50–100 candidates)	Embed on-demand at ranking time. 50 candidates × 500 tokens = 25K tokens = negligible cost. No pre-computation needed.
New JD pasted by user	Embed the new JD text using API immediately. Store result. Cost: 1 call, ~500 tokens = \$0.00001.

Prompt / Call Design

- Input: candidate_text string (same construction as local — title + summary + career descriptions + skills)
- Call: embed a batch of up to 100 texts per API call to reduce latency
- Output: list of float vectors. Normalize to unit length before storing (cosine similarity requires unit vectors)
- Dimensions: 1536 (OpenAI) or 768 (Google) — update .npy file shape accordingly

2.2 Module 9: Re-Ranker — LLM API Version

The cross-encoder is replaced by LLM API calls. The LLM reads the JD context and candidate summary together — exactly what a cross-encoder does, but with the full language understanding of a frontier model.

Prompt Design for LLM Re-ranking

Each API call processes 5 candidates (batch of 5). The prompt has three parts:

Prompt Part	Content	Token Budget
System prompt	You are a senior technical recruiter specializing in AI/ML engineering roles. You evaluate candidates rigorously and fairly. Return ONLY valid JSON. No extra text.	~80 tokens
JD context (user message)	Structured requirements: 6 skill groups with weights, 6 disqualifiers, ideal profile description (5-9 yrs, product company, production ML deployment, retrieval/ranking experience)	~200 tokens

Prompt Part	Content	Token Budget
Candidate summaries (user message)	5 candidate summaries: title + YoE + last 3 career descriptions (150 chars each) + top 6 skills + availability summary	~600 tokens (120 per candidate)
Output instruction	Rate each candidate 0-100. Return JSON array: [{id, score, reason, concern}]. reason = 1 sentence specific to their profile. concern = main gap or 'none'.	~60 tokens

Expected Response Format

The LLM must return strictly this JSON — validate before parsing:

- [
 - { "id": "CAND_0042871", "score": 87, "reason": "Built production FAISS index at Meesho serving 3M users.", "concern": "none" },
 - { "id": "CAND_0019884", "score": 72, "reason": "Strong NLP background but only 8-month notice period.", "concern": "Long notice period" },
- ... 3 more candidates
-]

Error Handling

Error	Detection	Handling
Invalid JSON response	json.loads throws exception	Retry once with temperature=0. If retry fails: use cross-encoder score (local fallback) for that batch.
Missing candidate ID	ID not in response	Use local score for that candidate. Do not fail the entire batch.
Score out of range	score < 0 or > 100	Clamp to [0, 100]. Accept the response.
Rate limit (429)	HTTP 429 response	Exponential backoff: wait 2^retry seconds. Max 3 retries. Then fall back to local.
API completely down	All retries fail	Fall back to cross-encoder (local) for all remaining batches. Log prominently.

2.3 Module 10: Reasoning Generator — LLM API Version

The biggest quality upgrade. Instead of template-based reasoning, the LLM reads the full candidate profile and writes grounded, specific, human-quality reasoning.

Prompt Design for Reasoning

Process 10 candidates per call (reasoning is shorter than re-ranking prompts):

- System: You are writing recruiter notes for a hiring manager. Be specific, honest, and concise. Reference exact data from the profile. 1-2 sentences maximum per candidate.
- User: For each candidate below, write a 1-2 sentence recruiter note explaining why they ranked where they did for a Senior AI Engineer role. Note: if they have concerns, mention them honestly.

- Candidate data: full profile dump (title, YoE, career descriptions, skills, location, availability, score, rank)
- Return JSON: `[[id, reasoning]]` — no preamble, no markdown

Grounding Enforcement (Critical)

After receiving LLM reasoning, run a grounding check before storing:

Claim Type in Reasoning	Grounding Check	Action if Fails
Title mentioned	Verify against <code>profile.current_title</code>	Replace with exact title from profile
Years of experience	Verify the number matches <code>profile.years_of_experience</code>	Replace with exact value
Company name	Verify company exists in <code>career_history</code>	Remove the company mention if not found
Skill mentioned	Verify skill exists in skills array with <code>trust > 0.3</code>	Remove skill mention if <code>trust < 0.3</code>
Location	Always append verified location from <code>profile.location</code>	Append even if LLM didn't mention it

2.4 Module 1: JD Parser — Dynamic LLM Version

The biggest new capability. Users can now paste ANY job description and the system extracts requirements dynamically using LLM — not just the hardcoded Redrob JD.

JD Parser Prompt

- System: You are a technical recruiter parsing job descriptions. Extract requirements precisely. Return only valid JSON.
- User: Parse this job description and extract: (1) required skills with weights summing to 1.0, (2) preferred skills, (3) disqualifying patterns with descriptions, (4) location preferences, (5) experience range (min, max, ideal), (6) 3 semantic search queries that would find ideal candidates
- Return JSON matching this exact schema: `{ hard_skills: [...], preferred_skills: [...], disqualifiers: [...], location_prefs: {...}, experience: {...}, semantic_queries: [...] }`

THIS UNLOCKS: The system now works for ANY job description — not just the hardcoded Senior AI Engineer role. Paste a Data Engineer JD, a Product Manager JD, a DevOps JD — the LLM extracts the right requirements every time. This transforms the system from a one-role tool into a general hiring platform.

3. New Features Enabled by API Mode

Feature 1 — Conversational Candidate Q&A

A chat interface where the user can ask natural language questions about the ranked candidates. The LLM has access to all ranked candidate data as context.

Example Question	What the LLM Does	Data It Uses
Why did candidate CAND_0042871 rank #3 instead of #1?	Compares score breakdowns of ranks 1, 2, and 3. Explains the specific differences in skill trust, career quality, and CE score.	features.skill_score, features.career_score, features.ce_score for all three candidates
Show me candidates with more than 24 months of FAISS experience	Filters skills array for FAISS with duration_months > 24 across all ranked candidates	skills array for all top-100
Which candidates are in Pune and can join in under 30 days?	Filters by location='Pune' AND notice_period_days < 30	profile.location, redrob_signals.notice_period_days
Summarize the top 10 candidates in one paragraph for my hiring manager	Generates an executive summary comparing the top 10 candidates' backgrounds and why they are strong picks	Full profile data for ranks 1-10
Which candidate has the best production deployment experience?	Scores all candidates on production_signal_score and returns the top candidate with specific evidence from their descriptions	career_history descriptions, production_signal_score from features

Feature 2 — Dynamic JD Input

Instead of being locked to the hardcoded Senior AI Engineer JD, users can paste any job description into a text area. The LLM parses it and the entire ranking pipeline runs for the new role.

- Use case: a recruiter wants to rank the same 100K candidates for a different role (Data Engineer, ML Platform Engineer, etc.)
- Implementation: text area in S3 JD Analyzer with a 'Parse with AI' button. LLM call replaces the rule-based JD parser.
- Output: same requirements JSON structure — the rest of the pipeline runs unchanged

Feature 3 — LLM Score Explanation Panel

When a judge or user wants to understand why the LLM scored a candidate a specific score, they can click 'Explain LLM Score' in S8. The system sends the candidate + JD to the LLM with a specific prompt: 'Explain in detail why this candidate received a score of N/100 for this role. Reference specific evidence from their profile.'

- Returns: a detailed 4-6 sentence explanation with specific evidence
- Shown in S8 as an expandable panel below the CE score display
- Different from the 1-2 sentence reasoning — this is a full analytical explanation

Feature 4 — Candidate Comparison

User selects two or three candidates from S7 (via checkboxes) and clicks 'Compare with AI'. The LLM receives all selected candidate profiles and produces a structured comparison.

Comparison Output	Content
Side-by-side strengths	What each candidate excels at specifically
Key differentiator	The one factor that separates the top candidate from the others
Recommendation	Which candidate to interview first and why
Risks	Specific concerns for each candidate (availability, skill gaps, seniority fit)

Feature 5 — Real-Time Ranking with Any JD

Combines Features 2 and 3. User pastes a new JD → LLM parses it → pipeline re-runs with new requirements → new top-100 produced. The system is now a general-purpose AI recruiter, not a one-role demo.

Feature 6 — AI Executive Summary

One-click button in S10 Export: 'Generate Executive Brief'. The LLM reads the top-20 candidates and produces a 300-word executive summary for a hiring manager. Exported alongside the CSV.

- Format: Introduction (role and pool size) + Top 3 Highlights (specific candidates) + Themes across top-20 + Recommended first interviews
- Shown in S10 as a preview panel. Downloadable as .txt or included in a combined .zip export

4. UI Changes — Screen by Screen

New Screen: S0 — API Settings

A new screen accessible from the sidebar navigation and from a settings icon in the navbar. This is where all API configuration lives.

Module	What It Contains
Provider Selection	Tab or dropdown: OpenAI Google Gemini Anthropic Claude Cohere. Selecting a provider shows the relevant API key field.
API Key Input	Masked text input per provider. 'Test Connection' button that makes a minimal API call to verify the key works. Shows green check or red error.
Per-Task Model Selection	Dropdown for each task: Embedding model, Re-ranking model, Reasoning model, JD Parsing model. Only shows models available from the selected provider.
Cost Estimator	Live estimate based on input size and selected models. Shows: 'Estimated cost for N candidates: \$X.XX'. Updates as user changes model selections.
Usage This Session	Running token count and cost for the current session. Helps users monitor spend.
Fallback Configuration	Toggle: 'If API fails, fall back to local models'. ON by default. When ON: any API failure silently uses local model. When OFF: API failure shows an error.
Mode Badge	Shows in the navbar: 'API Mode — Gemini 1.5 Flash' badge when API is configured and active.

Updated S2 — Home Dashboard

Change	What to Update
Architecture diagram	Update Step 5 label: 'Bi-Encoder Semantic (Local) or API Embedding'. Update Step 8 label: 'Cross-Encoder (Local) or LLM API Re-ranking'. Add Step 9: 'LLM Reasoning Generation (API Mode)'.
Design Decisions panel	Add a fifth card: 'Dynamic JD Parsing — Paste any job description and LLM extracts requirements in real time. Works for any role, not just Senior AI Engineer.'
Mode indicator	If API mode is active, show a purple banner below the hero: 'Running in API Mode — Gemini 1.5 Flash for re-ranking and reasoning. Quality is significantly enhanced.'

Updated S3 — JD Analyzer

Change	What to Add or Update
Dynamic JD Input box	New component at the top of S3: a collapsible panel with a large text area. Label: 'Paste a custom job description'. Button: 'Parse with AI'. Only visible when API mode is active.

Change	What to Add or Update
Parse comparison view	After LLM parsing, show two columns: 'Rule-based extraction' vs 'LLM extraction'. Highlights where they differ — judges see the LLM caught nuances the rules missed.
JD source badge	Small badge below the JD title: 'Original Hackathon JD' or 'Custom JD — parsed by GPT-4o' depending on which JD is active.
3 semantic queries display	Now shows which embedding model generated the queries: 'Queries generated by: text-embedding-004' vs 'Generated by: all-MiniLM-L6-v2 (local)'.

Updated S6 — Processing Tracker

Change	What Updates
Stage 8 label	In API mode: 'LLM Re-Ranking — Gemini 1.5 Flash'. In local mode: 'Cross-Encoder Re-ranking — ms-marco-MiniLM-L-6-v2'. Purple pulsing ring stays for both.
New Stage 9	Only in API mode: 'LLM Reasoning Generation'. Shows 'Generating reasoning for top 100 candidates via GPT-4o-mini'. Blue (not purple) since it's post-ranking.
Live counter Stage 8	In API mode shows: 'API batch N/20 complete (5 candidates each)' instead of CE batch counter.
Live counter Stage 9	Shows: 'Reasoning generated for N / 100 candidates'
Completion summary card	In API mode: Shows 'LLM Model: Gemini 1.5 Flash Reasoning Model: GPT-4o-mini Total tokens: ~45K Estimated cost: \$0.08'
Log panel	API mode log shows token counts per batch: '[12:34:05] Batch 3/20 complete — 1,240 tokens used (\$0.0002)'

Updated S7 — Results Table

Change	What Updates
CE Score column → LLM Score column	In API mode: column header becomes 'LLM Score'. Same purple badge treatment. But now shows GPT-4o-mini or Gemini 1.5 Flash score, not cross-encoder score.
Reasoning column	In API mode: reasoning is 2-3 sentences, LLM-generated, much richer than template. Still truncated in table but full on hover.
Row action menu	Right-click or three-dot menu on each row in API mode adds: 'Ask AI about this candidate', 'Compare with next candidate', 'Explain ranking'. Opens conversational panel.
Summary bar	Adds: 'LLM reasoning: GPT-4o-mini Re-ranked by: Gemini 1.5 Flash Session cost: \$0.08'

Updated S8 — Candidate Detail

Change	What Updates
--------	--------------

Change	What Updates
Reasoning section	In API mode: reasoning is LLM-generated (richer, longer). Shows 'AI Generated' purple badge + model name attribution.
LLM Evidence tab	In API mode: richer evidence — shows full prompt sent, full response received (truncated), token count, model used, response time.
New 'Ask AI' input	In API mode: a chat input box at the bottom of S8: 'Ask a question about this candidate'. User types a question, LLM answers using full profile as context. Example: 'Is this person likely to negotiate salary?' or 'What would their first 30 days look like?'
Score explanation button	Next to the LLM/CE score: 'Explain Score' button. Triggers a dedicated LLM call that explains in 4-6 sentences exactly why this score was given.

New Panel: Conversational Interface (S12)

A new screen accessible from sidebar: 'AI Chat'. A full conversational interface for exploring the ranking results.

Component	Description
Chat window	Standard chat interface: user messages on right, AI responses on left. Full chat history for the session.
Context selector	Dropdown above chat: 'Context: All 100 ranked candidates' or 'Context: Top 10 only' or 'Context: Specific candidate [ID]'. Controls how much data is in the LLM context window.
Suggested questions	4-6 clickable suggestion chips below the input: 'Why is #1 ranked above #2?', 'Which candidates are in Pune?', 'Summarize top-10 for my hiring manager', 'Who has the most FAISS experience?'
Data citations	Every AI response includes inline citations linking to the relevant candidate's S8 detail view — judge can verify the claim is real
Token counter	Small counter below the input: 'Session tokens used: 12,450 / 128,000 Estimated cost: \$0.02'

Updated S10 — Export

Change	What to Add
AI Executive Brief button	New button: 'Generate AI Executive Brief (GPT-4o-mini)'. Triggers LLM call, produces 300-word summary. Shown in preview panel, downloadable as .txt
Export options	In API mode: Download CSV (always available), Download CSV + Brief (.zip), Download Full Report (.docx with reasoning + brief + stats)
Session cost summary	Bottom of page: 'Total API cost this session: \$0.11 Tokens used: 68,400'

5. Cost Estimation — What Each Run Costs

5.1 Per-Run Cost by Provider

Task	Volume per 100K run	OpenAI Cost	Gemini Cost	Anthropic Cost
Embedding (API)	100K candidates × ~500 tokens = 50M tokens	\$1.00 (text-embed-3-small)	~\$0.00 (free tier covers this)	N/A — no embedding model
LLM Re-ranking	300 candidates in batches of 5 = 60 calls × ~1000 tokens = 60K tokens	\$0.009 (GPT-4o-mini)	\$0.0015 (Flash)	\$0.0075 (Haiku)
Reasoning generation	100 candidates in batches of 10 = 10 calls × ~800 tokens = 8K tokens	\$0.0012	\$0.0002	\$0.0040
JD parsing (one-time)	1 call × ~1500 tokens	\$0.0002	\$0.00003	\$0.0006
TOTAL per 100K run	—	~\$1.01	~\$0.002 to \$0.02	~\$0.012 (no embedding)

Task	Volume per Demo Run (50 candidates)	Cost (any provider)
Embedding 50 candidates	50 × 500 tokens = 25K tokens	< \$0.001 — negligible
LLM re-ranking 50 candidates	10 batches × 5 candidates × ~1000 tokens = 10K tokens	< \$0.002
Reasoning for 20 candidates	2 batches × 800 tokens = 1.6K tokens	< \$0.0003
Total per demo run	~37K tokens	< \$0.005 — less than half a cent

COST SUMMARY: A full 100K run with Google Gemini costs almost nothing (text-embedding-004 is free, Flash is near-free). A full 100K run with OpenAI costs about \$1. A demo run of 50 candidates costs less than half a cent with any provider. API mode is extremely affordable.

6. Fallback Strategy — When API Fails

Every API-powered component must fall back to the local equivalent. The system must never fail to produce a valid CSV because an API call failed.

API Component	Failure Scenario	Fallback Action	Quality Impact
API Embedding	Rate limit or API down	Use local sentence-transformers embedding on-the-fly. Slower but works.	Slight quality reduction — local model is smaller
LLM Re-ranking	API fails for a batch	Use cross-encoder (local) for that batch. Log which candidates used fallback.	Minimal — cross-encoder is still good quality
LLM Reasoning	API fails	Use template-based reasoning builder. Produces valid 1-2 sentence reasoning.	Reasoning less nuanced but still valid for submission
Dynamic JD Parsing	API fails	Fall back to rule-based JD parser on the hardcoded job_description.docx	Only impacts dynamic JD feature — core ranking works fine
Conversational Q&A	API fails	Show error in chat: 'AI Chat unavailable. API connection failed. Try refreshing or check S0 settings.'	Feature unavailable — does not impact ranking output
Executive Brief	API fails	Hide the 'Generate Brief' button. Show: 'AI Brief requires active API connection.'	Export feature reduced — CSV download still works

7. Implementation Order

Build in this sequence — each step builds on the previous one:

Step	What to Build	Time Estimate	Why This Order
1	S0 API Settings screen — key input, test connection, provider selection, model dropdowns	3–4 hours	All other API features depend on having keys configured and validated
2	LLM Reasoning Generator (API version) — replaces template builder	3–4 hours	Lowest risk — even if LLM reasoning fails, template fallback works. High visible quality impact.
3	LLM Re-ranker (API version) — replaces cross-encoder for top-300	4–5 hours	Medium risk — requires careful JSON parsing and batch management. Cross-encoder fallback available.
4	Update S6 Processing Tracker — new stage labels, Stage 9 for reasoning, cost display	2–3 hours	Visual update only — all logic is in backend. Do after backend works.
5	Update S7 Results Table — LLM Score column, richer reasoning in rows	2 hours	Visual update — depends on API response having llm_score and llm_reason fields
6	Update S8 Candidate Detail — richer reasoning panel, Ask AI input, explain score button	4–5 hours	Ask AI requires a new API call per question. Explain score requires a new API call.
7	Dynamic JD Parsing in S3 — text area, Parse with AI button, comparison view	3–4 hours	Standalone feature — doesn't affect ranking pipeline once built
8	S12 Conversational Interface — full chat screen with context selector	5–6 hours	Most complex new screen. Build last once all other API features are stable.
9	S10 Executive Brief — LLM call, preview panel, zip export	2–3 hours	Simple addition to existing export screen
10	Cost tracking throughout — token counters, session cost display	2 hours	Final polish — add to all screens after core features work

END OF API MODE IMPLEMENTATION GUIDE

4 providers · 6 new features · 8 updated screens · 1 new screen · Full fallback strategy